CareerLens AI

AI-Powered Career Intelligence Platform

Project Submission Report

Project Title	CareerLens AI
Domain	Artificial Intelligence / NLP / Full-Stack
Tech Stack	Python · Streamlit · Groq API · Sentence Transformers · Adzuna API

1. Executive Summary

CareerLens AI is an end-to-end AI-powered career intelligence platform built to solve a real, everyday problem: job searching is broken for most candidates. People upload their resumes to generic portals, receive no feedback on why they were rejected, have no idea what skills the market currently demands, and get no personalised guidance on where their career should go next.

This platform changes that. A user uploads their resume as a PDF. Within seconds, the system extracts their skills from a vocabulary of 6,732 industry terms, predicts which career roles best match their profile, fetches live job listings from the Adzuna Jobs API across 7 global markets, semantically ranks every job against the resume, and runs seven AI-powered analysis features — all powered by Meta's Llama 3.3-70B model via Groq's free LPU inference API.

The result is a product that functions like a personal career coach, a market intelligence analyst, and a resume consultant combined — built entirely on free and open-source infrastructure.

2. Problem Statement

The job search experience in 2026 is fundamentally misaligned with how hiring actually works. Candidates face three core problems:

2.1 The Resume Black Hole

Most job applications receive no response. Candidates have no visibility into whether their resume passed ATS screening, how their skills compare to what the market wants, or why they were rejected. There is no feedback loop.

2.2 Skills vs. Market Mismatch

Job seekers typically have no real-time data on which skills are currently in high demand, which are declining, or what specific gaps exist between their profile and the roles they are targeting. Generic job boards list vacancies but provide no intelligence.

2.3 Lack of Personalised Guidance

Career progression, salary benchmarking, interview preparation, and resume optimisation are all addressed by separate, disconnected tools — many of which require expensive subscriptions. There is no single platform that handles the full career intelligence lifecycle in one place, for free.

CareerLens AI was built directly in response to these three gaps.

3. Objectives

The project was scoped with the following clear objectives:

- Build a fully working, end-to-end career intelligence platform — not a demo or prototype
- Parse and understand real PDF resumes with production-grade accuracy
- Extract skills intelligently from a large-scale, curated vocabulary rather than naive keyword matching
- Predict the most likely career roles for a candidate using semantic similarity, not just title matching
- Fetch real-time job listings from the live market and rank them semantically against the candidate's profile
- Deliver seven distinct AI-powered career features backed by a large language model
- Build the entire system on free and open-source infrastructure — no paid API requirements for the core pipeline
- Design a professional, enterprise-quality user interface despite using Streamlit as the frontend framework

4. Approach & Methodology

The project was built in two phases. Phase 1 established a robust, production-grade backend pipeline. Phase 2 added the AI intelligence layer and multi-page user interface.

4.1 Phase 1 — Core Pipeline

The initial codebase was a flat Python project with no proper structure, broken imports, and several fundamental bugs. The first task was a full audit across all 20+ source files to identify and categorise every failure point.

Six critical bugs were identified and fixed:

- Skills extraction was matching single characters and generic words with no vocabulary validation
- Experience detection was reading all years in the document (including education dates) rather than isolating work history sections
- Role prediction had no fallback logic and produced irrelevant results when embeddings were weak
- Job matching scores were on an uncalibrated scale, making ranking unreliable
- The Adzuna API client had no retry logic, no key rotation, and crashed on rate limits
- Skill gap analysis had no synonym handling, causing false negatives for semantically equivalent skills

4.2 Phase 2 — AI Intelligence Layer

With the pipeline stabilised, seven AI features were added on top of the core engine. Each feature calls a large language model with a carefully structured system prompt that enforces a strict JSON output schema. This ensures consistent, parseable responses regardless of what the model returns.

The seven features are: Resume Scorer, AI Job Fit Analyser, Cover Letter Generator, Salary Intelligence, Career Path Planner, Resume Bullet Rewriter, and Interview Coach. Each is covered in detail in Section 6.

4.3 Semantic Matching Architecture

Job ranking uses a three-component weighted scoring formula rather than simple keyword overlap:

Component	Weight	What It Captures
Semantic Similarity (embeddings)	50%	Conceptual alignment between resume and job
Skill Overlap Ratio	35%	Exact and near-exact skill matches
Skill Density Bonus	15%	Job listings with concentrated skill mentions

5. Tools & Technologies Used

Every tool in this stack was chosen deliberately. The rationale for each is given below.

Tool / Library	Category	Reason for Choosing It
Python 3.11	Language	Dominant language for ML/NLP tooling. Best library ecosystem for the pipeline we needed.
Streamlit	Frontend	Allows rapid development of data-heavy UIs in pure Python. No JavaScript required, yet capable of a professional look with CSS injection.
Groq API + Llama 3.3-70B	LLM Inference	Groq runs Meta's Llama 3.3-70B on custom LPU hardware at ~600 tokens/second. It is free (14,400 requests/day), significantly faster than cloud LLM APIs, and demonstrates knowledge of open-source LLM infrastructure — a more impressive portfolio choice than simply calling a paid API.
Sentence Transformers (MiniLM-L12-v2)	Embeddings	Multilingual, lightweight, and runs on CPU. Produces 384-dimensional embeddings suitable for semantic similarity at the scale of job listings (hundreds to thousands of documents).
Adzuna Jobs API	Live Job Data	One of the few job APIs with genuine multi-country coverage (India, US, UK, Canada, Australia, Germany, Singapore), structured JSON responses, and a free developer tier. LinkedIn and Naukri do not offer public APIs.
pdfplumber	PDF Parsing	Preserves text layout and column structure better than PyPDF2 or pdfminer alone. Correctly handles multi-column resumes where other libraries merge lines from different columns.
scikit-learn (KMeans)	Clustering	Used to cluster live job titles from the market into representative role labels. Lightweight and fast on CPU for the scale of data involved (hundreds of job titles).
ESCO v1.1 Dataset	Skills Data	The European Commission's official occupational skills ontology. 126,051 occupation-skill mappings, 13,890 distinct skills, multilingual. Using ESCO gives the skills vocabulary academic and institutional credibility that a hand-curated list cannot match.
asyncio + httpx	Async HTTP	Fetching jobs for 5+ roles in parallel rather than sequentially reduces total API time from ~25 seconds to ~6 seconds. httpx supports async out of the box, unlike the requests library.
python-dotenv	Config	Keeps API keys out of source code. Standard practice for any project intended for version control.

6. Datasets

6.1 ESCO v1.1 (European Skills Ontology)

Source: European Commission — skills.europa.eu

Files used: occupationSkillRelations_en.csv · skills_en.csv · occupations_en.csv

Scale: 126,051 occupation-skill relationships · 3,043 occupations · 13,890 skills

How it was used: Used to build the core skills vocabulary and role-to-skill mapping. Each skill's preferred label and alternative labels were extracted to maximise matching coverage.

ESCO was chosen over a hand-curated list because it is maintained by a recognised institution, is updated regularly, covers 27 languages (future-proofing for multilingual resumes), and carries occupational taxonomy metadata that can support career path reasoning.

6.2 IT Job Roles & Skills Dataset (Kaggle)

Source: Kaggle — IT_Job_Roles_Skills.csv

Scale: 493 IT job titles with associated required skills and certifications

How it was used: Merged with ESCO to add modern IT-specific roles and skills (Kubernetes, Terraform, Groq, LLM-related terms) that are not yet in the official ontology. Final merged vocabulary: 6,732 skills, 3,367 role profiles.

6.3 Live Job Data (Adzuna API)

Source: Adzuna Jobs API — api.adzuna.com

Scale: Up to 100 listings per role query, across 7 countries

How it was used: Fetched in real-time on each user session. Used for job ranking, market heatmap construction, salary intelligence extraction, and role discovery clustering.

7. System Architecture

The system is organised as a modular backend with a single Streamlit frontend entry point. The architecture separates concerns cleanly across seven backend modules.

Module	Responsibility
resume_processing/	PDF text extraction (pdfplumber), skills extraction (6,732-term vocab), experience detection (section-isolated, month-summed)
role_inference/	3-tier role prediction: semantic embedding similarity → skill-cluster scoring → keyword heuristic fallback
job_fetching/	Async parallel Adzuna API calls, smart query construction, API key rotation, exponential backoff on 429s, deduplication
matching/	Semantic job ranking using weighted 3-component score (50/35/15). Embedding + skills cache to avoid redundant computation.
analysis/	Synonym-aware skill gap analysis returning strong_match, partial_match, and missing skill lists per job
market_intelligence/	Market heatmap from job descriptions, KMeans role discovery, market strength scoring, skill ROI simulation
ai_features/	7 LLM-powered features via Groq API. Strict JSON schema enforcement in system prompts. Fallback to local flan-t5-base if API unavailable.
semantic/	Embedding engine wrapping sentence-transformers with MD5-keyed in-memory cache to avoid re-encoding identical texts

8. Features & Output

The platform is a multi-page Streamlit application with seven navigation pages, each delivering a distinct capability.

8.1 Core Pipeline Output (all pages)

- Skills extracted from resume with 6,732-term vocabulary — displayed as tagged chips
- Career role predictions — top 5 roles semantically matched to the candidate profile
- Live job listings fetched in real time — up to 100 per role across the selected country
- Semantic job ranking — every job scored 0–100 and ranked by relevance

8.2 AI-Powered Features

Feature	What It Delivers
Resume Scorer	Five-dimension score (Impact, Clarity, ATS Compatibility, Skills, Structure). Overall 0–100 rating. Top 3 strengths, 3 critical fixes, missing ATS keywords. Score visualised as a circle gauge with dimension bars.
AI Job Fit	Per-job fit score and verdict (Strong Match / Good Fit / Partial Fit / Stretch Role). Lists why you fit, why you might struggle, a standout angle unique to that role, and targeted application advice.
Cover Letter	Personalised 3-paragraph letter referencing the actual company and job title. Three tone options: Professional, Conversational, Bold. Downloadable as a .txt file.
Salary Intelligence	Salary floor, median, and ceiling plus p25–p90 percentiles extracted from live job listing data. Local currency (INR / USD / GBP / EUR). Negotiation tips and market context summary.
Career Path Planner	Three-step career progression roadmap: current level → near term → medium term. Each step includes timeline, skills to add, recommended certifications, and average salary. Immediate / short-term / long-term skills roadmap.
Resume Bullet Rewriter	Identifies the five weakest bullet points in the resume and rewrites each with action verbs and quantified impact. Before/after comparison grid. Full rewritten resume downloadable.
Interview Coach	Four technical questions with model answers (difficulty-tagged Easy/Medium/Hard) and three behavioural questions using the STAR framework. Questions to ask the interviewer. Preparation tips and red flags to avoid.

9. Challenges Faced

9.1 Skills Extraction Accuracy

The naive first implementation matched individual words against the skills list, producing hundreds of false positives (common English words like 'analysis', 'management', 'design' were flagged as skills). The fix was a dual layer of validation: a GENERIC_TERMS blocklist of 60+ noise words, combined with requiring phrases to be present in the 6,732-term curated vocabulary. Tri-gram scanning was added so multi-word skills like 'infrastructure as code' or 'machine learning algorithms' are correctly matched as single units.

9.2 Experience Detection Reliability

Early implementation parsed all four-digit years anywhere in the document and subtracted min from max. This was wildly inaccurate — a Computer Science graduate who studied from 2018–2022 would appear to have 4 years of work experience. The solution was section isolation: the parser identifies section headers ('Work Experience', 'Employment History', etc.) and only scans that portion of the document. Month-level date parsing was also added to prevent rounding errors.

9.3 LLM Output Reliability

Large language models occasionally return malformed JSON — extra trailing text, markdown code fences, or nested responses. A robust _parse_json() function was written that strips markdown fences with regex, then uses bracket-depth scanning to find the outermost valid JSON object or array regardless of what surrounds it. This eliminated all parsing failures in testing.

9.4 Job API Rate Limits

Fetching jobs for 5+ roles simultaneously caused frequent 429 (Too Many Requests) responses from the Adzuna API. Two strategies were combined: API key rotation using itertools.cycle across multiple keys, and exponential backoff on 429 responses. Async fetching with asyncio.gather also meant that individual failures do not block the entire request batch.

9.5 Streamlit Sidebar Navigation

Streamlit's native st.radio() widget renders as a list of radio buttons with selection circles — visually incompatible with a professional UI. The sidebar navigation was rebuilt using st.button() with CSS injection to override all Streamlit button styling: transparent background, left border active indicator, uppercase DM Mono font at 9px, and white-space: nowrap to prevent text wrapping. Session state was used to persist the active page across reruns.

9.6 Free Infrastructure Constraints

Building on entirely free infrastructure meant working within real limitations. The Groq free tier has a daily request limit of 14,400. The Adzuna API has per-second rate limits. The embedding model runs on CPU. Each of these constraints was handled architecturally: in-memory caching of embeddings, API key rotation, request batching, and a local flan-t5-base fallback for the LLM if Groq is unavailable.

10. Future Scope

- User authentication and profile persistence — save analyses, track skill progress over time
- Support for additional job markets (UAE, Netherlands, France) via additional Adzuna country codes
- Integration with LinkedIn API (when accessible) to directly compare profile against market without PDF upload
- Resume version control — store multiple versions and compare AI scores across iterations
- Skill trend tracking — show how demand for a given skill has changed over the past 3, 6, 12 months using historical job data
- Employer-side interface — companies could upload job descriptions and receive a ranked shortlist of candidate profiles
- Fine-tuned embedding model on resume and job description data for improved semantic matching accuracy
- Mobile-responsive progressive web app (PWA) wrapper around the Streamlit application

11. Conclusion

CareerLens AI demonstrates that a sophisticated, production-grade AI product can be built on entirely free and open-source infrastructure. The system processes real PDF resumes, fetches live job data, applies semantic matching using pre-trained language models, and delivers seven distinct AI-powered career intelligence features — all from a single Python application.

The decisions made throughout the build reflect real engineering judgement: choosing Groq over a paid API because of speed and portfolio differentiation, choosing ESCO over hand-curated data because of institutional credibility, building async job fetching because sequential requests were too slow, and designing the AI output schema enforcement because LLMs produce inconsistent output without it.

The product is ready for live demonstration and can be run locally with three lines of setup. All sensitive credentials are managed through environment variables and excluded from version control.